



1. Problem Title & Link

Title: LeetCode 1189 – Maximum Number of Balloons

Link: <https://leetcode.com/problems/maximum-number-of-balloons/>

2. Problem Statement (Short Summary)

Given a string text, determine the maximum number of times you can form the word "**balloon**" using the characters in the string.

Each character can be used **only once**.

Return the maximum number of complete "balloon" words that can be formed.

3. Examples (Input → Output)

Example 1

Input

text = "nlaebolko"

Output

1

Explanation

One complete "balloon" can be formed.

Example 2

Input

text = "loonbalxballpoon"

Output

2

Example 3

Input

text = "leetcode"

Output

0

4. Constraints

- $1 \leq \text{text.length} \leq 10^4$
- text consists of lowercase English letters only.

5. Core Concept (Pattern / Topic)

- HashMap / Frequency Counting



- Character Counting
- Greedy

6. Thought Process (Step-by-Step Explanation)

Brute Force

Repeatedly try to build the word "balloon":

- Search for each required character.
- Remove it from the string.
- Count how many complete words can be formed.

Time Complexity:

$O(n \times \text{answer})$

This involves unnecessary repeated scans.

Optimized Thinking

Instead of repeatedly forming the word, count the frequency of every character once.

The word "**balloon**" requires

b : 1

a : 1

l : 2

o : 2

n : 1

The answer is simply the minimum among

`count('b')`

`count('a')`

`count('l') // 2`

`count('o') // 2`

`count('n')`

Intuition

Example

`text = "loonbalxballpoon"`

Character counts

b = 2

a = 2

l = 4

o = 4



$n = 2$

Since

l and o are needed twice

Effective counts become

$l = 2$

$o = 2$

Minimum = 2

↓

Two "balloon" words can be formed.

Why This Works

A complete word is limited by the character that is available the fewest times after considering its required frequency.

7. Visual / Intuition Diagram (ASCII)

Required

balloon

b ×1

a ×1

l ×2

o ×2

n ×1

↓

Available

b = 2

a = 2

l = 4

o = 4

n = 2

↓

2 balloons

8. Pseudocode (Language Independent)

```
count frequency of every character
```

```
answer = minimum(
```



```
count['b'],  
  
count['a'],  
  
count['l'] / 2,  
  
count['o'] / 2,  
  
count['n']  
  
)  
  
return answer
```

9. Code Implementation

Python

```
from collections import Counter  
  
class Solution:  
    def maxNumberOfBalloons(self, text: str) -> int:  
  
        count = Counter(text)  
  
        return min(  
            count['b'],  
            count['a'],  
            count['l'] // 2,  
            count['o'] // 2,  
            count['n']  
        )
```

Java

```
class Solution {  
  
    public int maxNumberOfBalloons(String text) {  
  
        int[] count = new int[26];  
  
        for (char ch : text.toCharArray()) {  
            count[ch - 'a']++;  
        }  
  
        return Math.min(  
            Math.min(count['b' - 'a'], count['a' - 'a']),  
            Math.min(  
                Math.min(count['l' - 'a'] / 2, count['o' - 'a'] / 2),  

```



```

        count['n' - 'a']
    )
    );
}
}

```

10. Time & Space Complexity

Time Complexity

$O(n)$

One pass through the string.

Space Complexity

$O(1)$

The frequency array has a fixed size of 26.

11. Common Mistakes / Edge Cases

- Forgetting that 'l' appears twice.
- Forgetting that 'o' appears twice.
- Using floating-point division instead of integer division.
- Returning the maximum instead of the minimum.
- Not handling missing characters (frequency = 0).

12. Detailed Dry Run

Example

text = "loonbalxballpoon"

Character frequencies

Character	Count	Required	Effective Count
b	2	1	2
a	2	1	2
l	4	2	2
o	4	2	2
n	2	1	2

Answer

$\min(2,2,2,2,2) = 2$

Output

2

13. Common Use Cases (Real-Life / Interview)



- Inventory management
- Recipe ingredient checking
- Resource allocation
- Character frequency analysis
- Anagram and word construction problems

14. Common Traps (Important!)

- Ignoring duplicate letters in the target word.
- Using / instead of integer division (// in Python).
- Counting all letters but forgetting to divide 'l' and 'o'.
- Trying to repeatedly construct the word instead of using frequencies.

15. Builds To (Related LeetCode Problems)

- LC 383 – Ransom Note
- LC 242 – Valid Anagram
- LC 387 – First Unique Character in a String
- LC 451 – Sort Characters By Frequency
- LC 1160 – Find Words That Can Be Formed by Characters

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Repeated Word Construction	$O(n \times \text{answer})$	$O(n)$	Inefficient
HashMap Frequency Count	$O(n)$	$O(1)$	Simple and readable
Frequency Array	$O(n)$	$O(1)$	Fastest and preferred

17. Why This Solution Works (Short Intuition)

A "balloon" can only be formed as many times as its least available required character. By counting character frequencies once and adjusting for characters that appear twice ('l' and 'o'), the minimum effective count directly gives the maximum number of "balloon" words that can be formed.

18. Variations / Follow-Up Questions

1. Generalize the solution for any target word.
2. Return the leftover characters after forming the maximum number of words.
3. Given multiple target words, determine which can be formed the most times.
4. Handle Unicode characters instead of lowercase English letters.



5. Extend the problem to support character updates in a streaming input.
- 6.